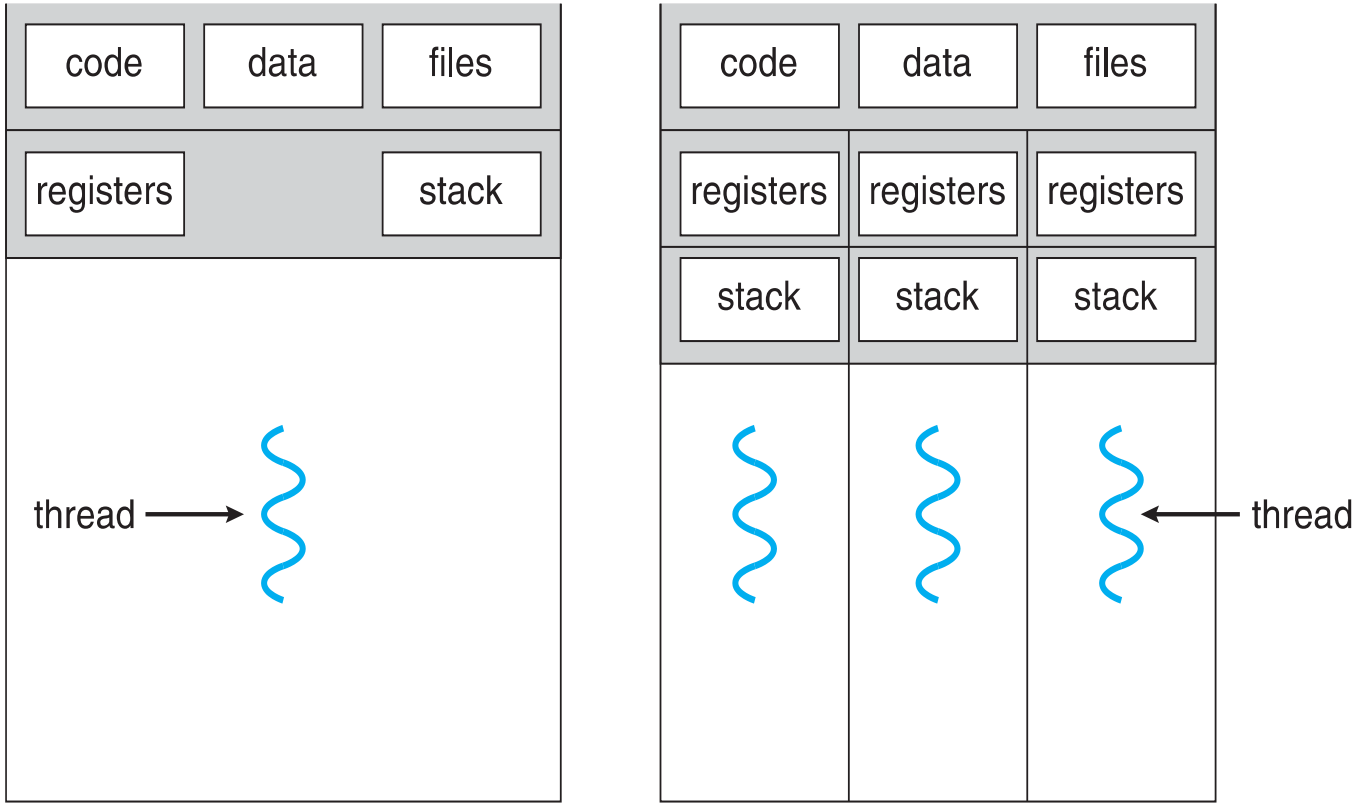
Threads

Introduction

- A thread is a basic unit of CPU utilization -a thread ID, a program counter, a register set, and a stack.
- It shares with other threads belonging to the same process its code section, data section, and other operating-system resources.
- A traditional (or heavyweight) process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time.



single-threaded process

multithreaded process

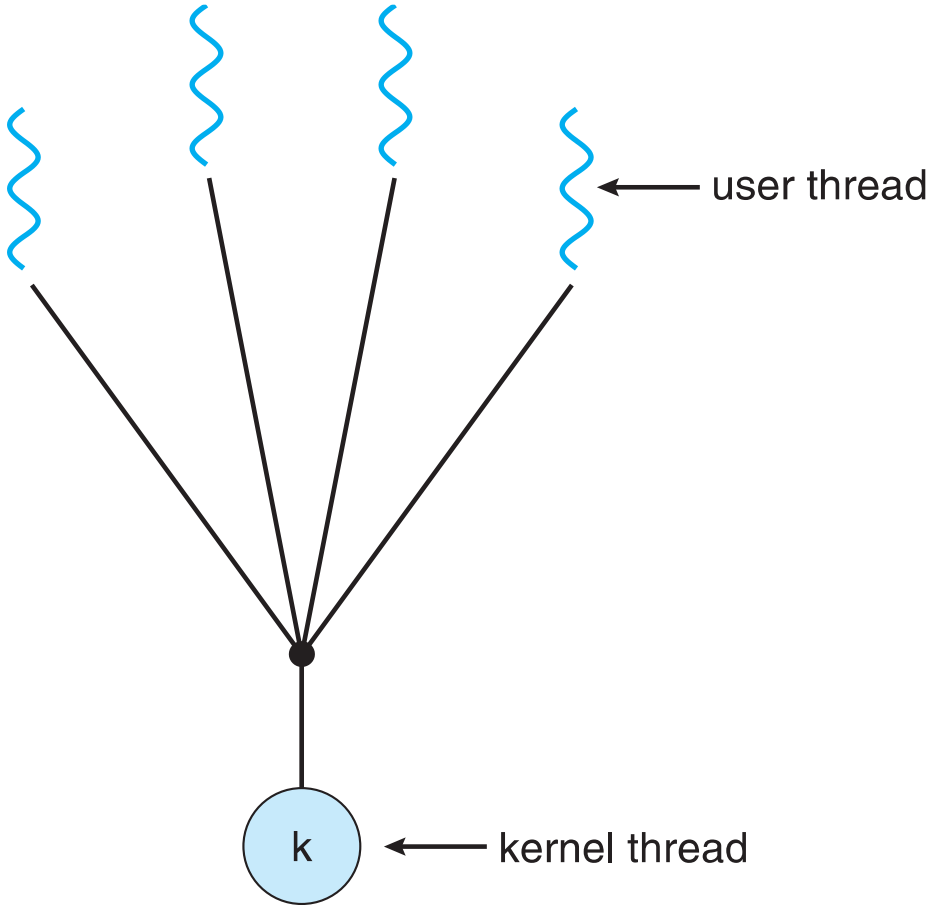
- Most modern applications are multithreaded
- Threads run within application
  - Multiple tasks with the application can be implemented by separate threads
  - A program may consist of many tasks that can run concurrently.
  - A thread is the flow of execution, from beginning to end, of a task.
  - A thread is essentially an object that facilitates the execution of a task.
- Process creation is heavy-weight while thread creation is light-weight
  - Threads are lightweight processes, as the overhead of switching between threads is less.
- Can simplify code, increase efficiency
- Kernels are generally multithreaded
- Benefits:
  - Responsiveness
  - Resource Sharing
  - Economy
  - Scalability

User Threads and Kernel Threads

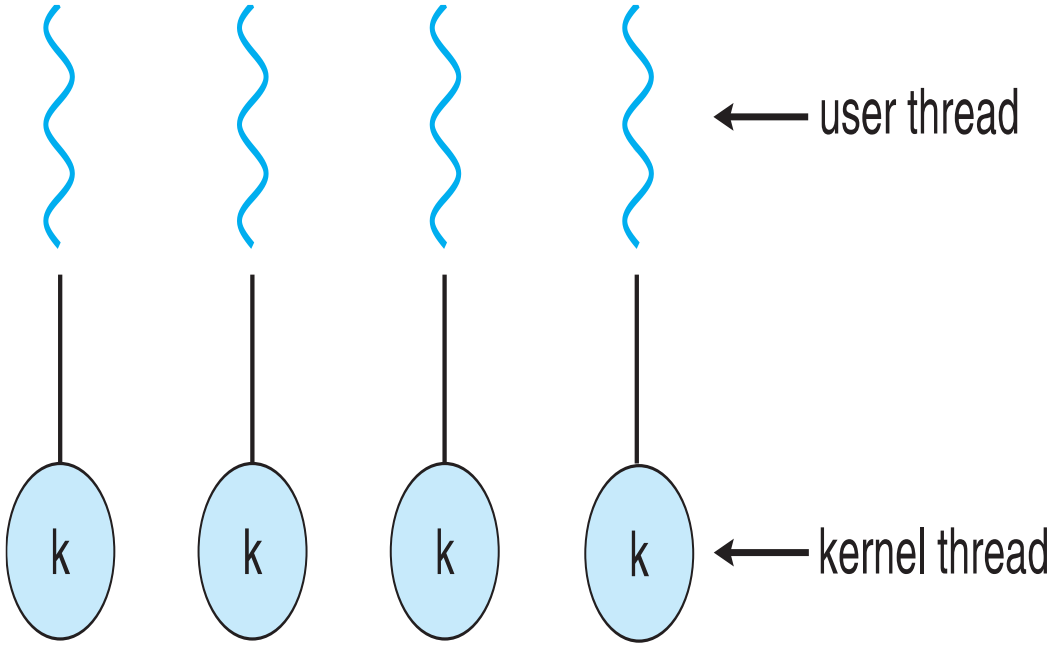
- **User threads** - User threads are supported above the kernel and are managed without kernel support.
- **Kernel threads** - Kernel threads are supported and managed directly by the operating system.
- Examples – virtually all general purpose operating systems, including:
  - Windows
  - Solaris
  - Linux
  - Mac OS X
- **A relationship must exist between user threads and kernel threads.**

Multithreading Models

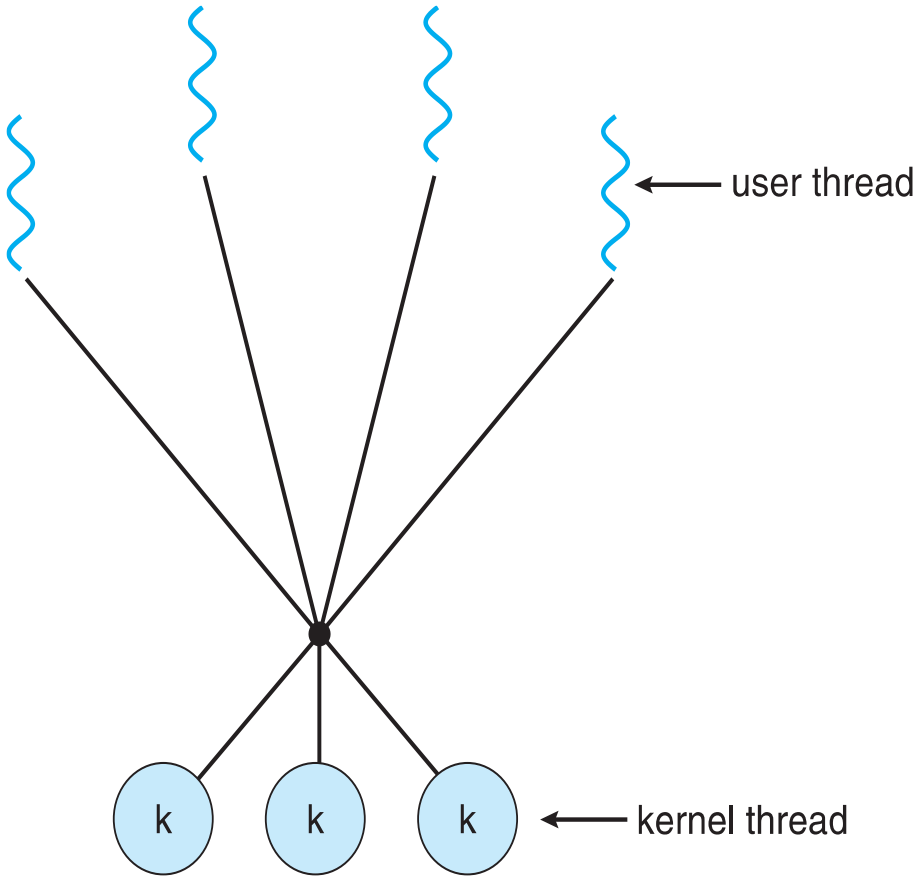
Many-to-one



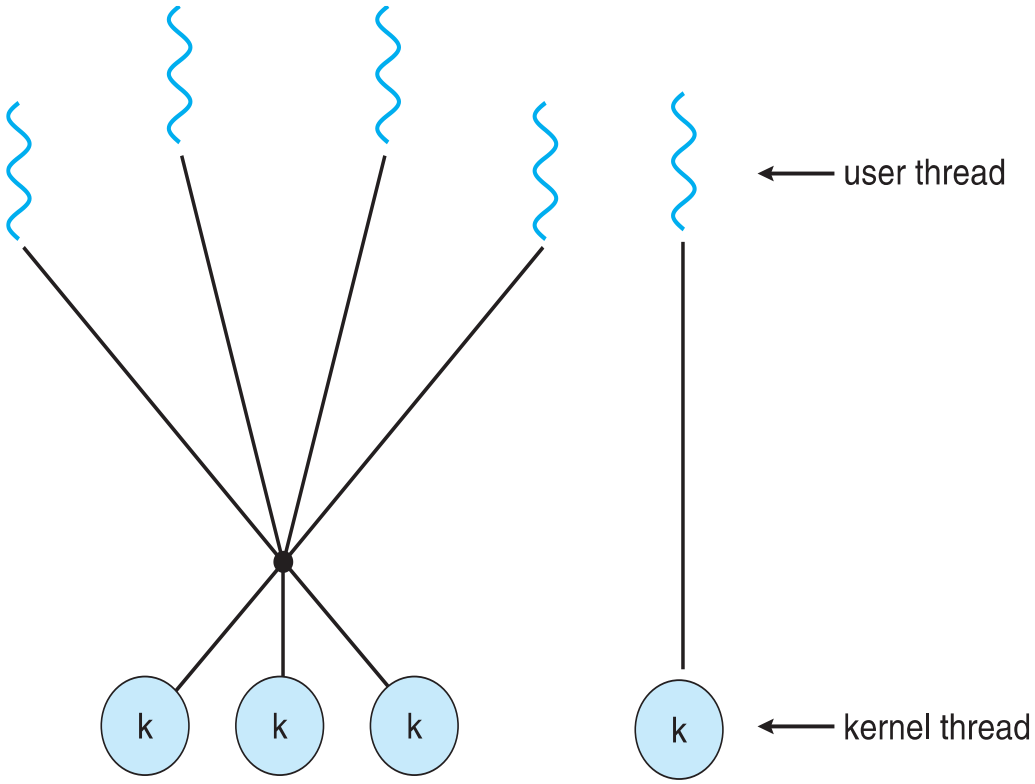
One-to-one



Many-to-Many



Two-level Model



Thread Libraries

- **Thread library** provides programmer with API for creating and managing threads:

- Two primary ways of implementing:
  - Library entirely in user space, with no kernel support
    - All code and data structures for the library exist in user space.
    - Invoking a function in the library results in a local function call in user space and not a system call.
  - Kernel-level library supported by the OS
    - Code and data structures for the library exist in kernel space.
    - Invoking a function in the API for the library typically results in a system call to the kernel.

- Three main thread libraries are in use today:
  - POSIX Pthreads
    - Pthreads, the threads extension of the POSIX standard, may be provided as either a user-level or a kernel-level library.
  - Windows
    - Windows thread library is a kernel-level library available on Windows systems.
  - Java
    - Java thread API allows threads to be created and managed directly in Java programs.
    - However, because in most instances the JVM is running on top of a host operating system, the Java thread API is generally implemented using a thread library available on the host system.

Java Threads

- Java threads are managed by the JVM
- Typically implemented using the threads model provided by underlying OS
- With Java, you can launch multiple threads from a program concurrently.

Creating threads

- In java threads can be created by extending the Thread class or implementing the Runnable Interface
- **Extend java.lang.Thread class**

OR

Implement java.lang.Runnable interface

- It is more preferred to implement the Runnable Interface so that we can extend properties from other classes

  - run() method must be overridden
  - run() is called when execution of the thread begins
  - A thread terminates when run() returns
  - start() method invokes run()
  - Calling run() does not create a new thread

- https://docs.oracle.com/javase/7/docs/api/java/lang/Runnable.html
- https://docs.oracle.com/javase/7/docs/api/java/lang/Thread.html

Define a taskclass by implementing the Runnable interface

```
public class thread1 {
   public static void main(String args[])
   {
   taskclass t1 = new taskclass(); //create tasks
   Thread m1 = new Thread (t1); //create threads
   m1.start(); // starts the thread by running the run method
   } }
class taskclass implements Runnable   //declare task class
{
   @Override
   public void run()     //override run() method to tell the system what task to perform
   {
      System.out.println("thread started");
   }  }
```

Define a taskclass by extending the Thread class

```
public class thread1 {
   public static void main(String args[])
   {
   taskclass t1 = new taskclass(); //create tasks
   t1.start(); // starts the thread by running the run method
   }
}
class taskclass extends Thread //declare task class
{
   @Override
   public void run()    //override run() method to tell the system what task to perform
   {
      System.out.println("thread started");

   } }
```

Example

Use the Runnable Interface to Create and Launch Threads

Objective: Create and run two threads:

1. The first thread prints the integers 0 through 4.
2. The second thread prints the integers 5 through 9.

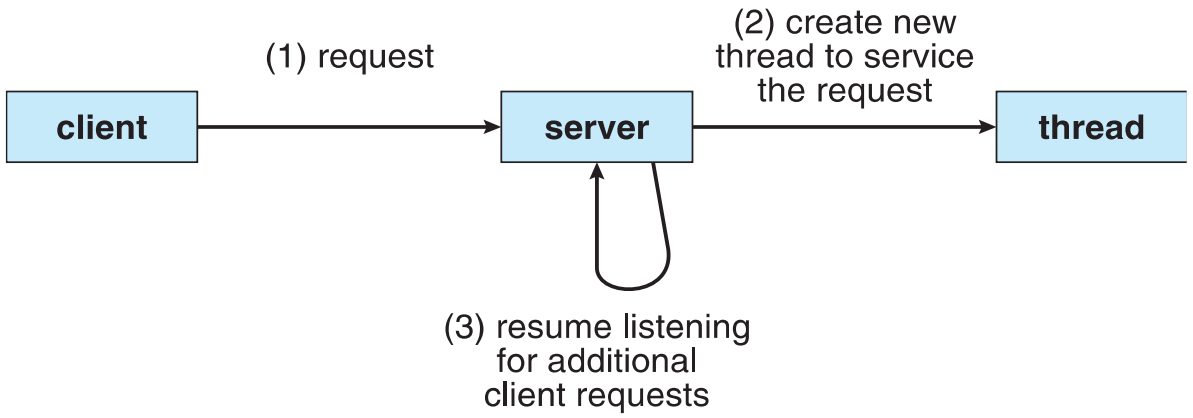
```
run:
thread1:0
thread2:5
thread2:6
thread2:7
thread1:1
thread1:2
thread1:3
thread1:4
thread2:8
thread2:9
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example: Using the Runnable Interface to Create and Launch Threads

- Objective: Create and run three threads:
  - The first thread prints the letter *a* 100 times.
  - The second thread prints the letter *b* 100 times.
  - The third thread prints the integers 1 through 100.

```
run:
01234bbbbbaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbb5678910111213141 5bb
baaaaaaaaaabbbbbbbbbbbbbbbb16bbbbbbbaaaaaaaaaaaaaaaaaaaaaaaaaaaaabbbbbbbbb17bbbbbbbbbbbbbbbaaaaa
aaaaaaaaaaaaaaaaabbbbbbbbbbbbbbbbbbbbbb181920212223242526272829303 1bbbbbaaabbbbbbbbbbbbbbbbb
3233bbbbaaaaaaaabbbbbbbbbbbbbbbbbbbb34bbbbbbbbbbbbbbbbbbbaaaaaaaaaaaaaaaaaaaaabbbbb3536373839b
bbbbbbbbbbbbbbbbbbbbbbbbbbbbbabbbbaa404142aaaaaaaa43aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa444
5464748495051525 3aaaa5455aaaa5657aaaaaaaaaaaaa5859606162636465666768697071727374757677787980
818283848586878889909192939495969798 99BUILD SUCCESSFUL (total time: 0 seconds)
```

Thread Pools



- Server applications process a large number of short tasks that arrive from some remote client.
- Disadvantages of the thread-per-request approach:
  - Overhead of creating a new thread for each request is significant
  - Creating too many threads in one JVM can cause the system to run out of memory or thrash due to excessive memory consumption.
- One solution to this problem is to use a **thread pool**.
  - To create a number of threads at process start and place them into a pool, where they sit and wait for work.
  - Number of threads in the pool?

Thread Pool - Example

```
import java.util.concurrent.Executor;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class thpool {
   public static void main(String args[])
   {
     ExecutorService executor = Executors.newFixedThreadPool(2); //create executor
     for (int i=0;i<5;i++)  //task queue
     {
        executor.execute(new task());  //submit task
     }
     executor.shutdown();       //shutdown executor
   }
}
class task implements Runnable //declare task
```

Reference

- "Operating System Concepts", Silberschatz, Galvin, Gagne, 9th Edition, Wiley India.
- Introduction to Java Programming: Comprehensive Version, Y. Daniel Liang, Pearson.